# CSC490 Assignment 2 – Data Processing Pipelines

Vaidik Patel (1008071159), Hardik Shah (1009453690), Paarth Arya (1008712564)

## Aspirational Datasets

1. TikTok Content & Exposure

**Purpose:** Minimal spine of what a consenting user saw/did + basic video metadata.

| Field | Type | Explanation |
|---|---|---|
| event_id | string | Unique event row ID |
| user_pseudo_id | string | Hashed user ID |
| timestamp | datetime | When the impression happened |
| video_id | string | TikTok video identifier |
| watch_time_ms | int | Milliseconds watched |
| like | bool | User liked the video |
| not_interested | bool | User marked “not interested” |
| impression_source | string | fyp (for you page) / following / search / hashtag / other |
| caption | string | Video caption text |
| hashtags | string | Comma-separated hashtags |
| duration_ms | int | Video duration (ms) |
| creator_id | string | Creator account ID |

2. Content Extracts (ASR + Segments + OCR)

**Purpose:** One place for the core analyzable signals (speech, scene ranges, on-screen text).

| Field | Type | Explanation |
|---|---|---|
| extract_id | string | Unique extract row ID |
| video_id | string | Source video |
| segment_start_ms | int | Segment start (ms) |
| segment_end_ms | int | Segment end (ms) |

| | | |
|---|---|---|
| asr_text | string | Transcript for this segment |
| asr_confidence | float | 0–1 confidence |
| ocr_texts | string | Comma-separated on-screen texts in segment |
| keyframe_uri | string | URL/path to representative frame |
| lang | string | Language code (e.g., “en”) |

3. Claims & Fact-Check Links

**Purpose:** Extracted statements (from ASR/OCR), normalized, with best available fact-check.

| Field | Type | Explanation |
|---|---|---|
| claim_id | string | Unique claim ID |
| video_id | string | Source video |
| source | string | asr / ocr |
| segment_range_ms | string | “start_ms-end_ms” for location |
| claim_text | string | Raw claim text |
| canonical_text | string | Normalized/paraphrased claim |
| checkworthy_score | float | 0–1 likelihood to fact-check |
| factcheck_url | string | Link to supporting fact-check |
| factcheck_verdict | string | true / false / misleading / unverified / mixture |

4. Authenticity / Manipulation Labels

**Purpose:** Ground truth for deepfake/manipulation detection (face, voice, whole-frame).

| Field | Type | Explanation |
|---|---|---|
| label_id | string | Unique label row ID |
| video_id | string | Source video |
| modality | string | face / voice / frame |
| label | string | real / synthetic / edited / unknown |
| manipulation_type | string | faceswap / lip_sync / voice_clone / fill / splice |

| | | |
|---|---|---|
| time_range_ms | string | “start_ms-end_ms” (optional) |
| annotator_confidence | float | 0–1 confidence |

5. User Feedback

**Purpose:** Close the loop: measure usefulness and refine models/explanations.

| **Field** | **Type** | **Explanation** |
|---|---|---|
| feedback_id | string | Unique feedback row ID |
| user_pseudo_id | string | Hashed user ID |
| item_ref | string | video_id or claim_id |
| item_type | string | video / claim |
| timestamp | datetime | When feedback was given |
| feedback | string | thumb_up / thumb_down / correct / incorrect |
| note | string | Optional free-text |
| expert_label | string | real / fake / misleading / n/a (optional) |

# Reality Check

In addition to relevant datasets for deepfake detection, we also include open-source tools that we will be using across our pipelines for separate use-cases, including fake news detection.

| Relevant Item | Description | Commentary |
|---|---|---|
| FaceForensics++ | Benchmark of manipulated face videos and originals, generated with four state-of-the-art face swap methods (DeepFakes, FaceSwap, Face2Face, NeuralTextures). Over 1.8 million frames from ~1,000 videos are provided for deepfake detection research. | Supports **authenticity labels** (real vs fake face content) in the pipeline. It's a **video modality** dataset widely used to train and evaluate deepfake detectors. Available for research use (we have received access via a sign-up form). **License/ToS:** Research-only; contains facial content of actors with consent. *Limitation:* Focuses on face swaps in controlled videos. TikTok videos may involve other manipulations or AI-generated audio. |
| Deepfake Detection Challenge (DFDC) | Meta's massive deepfake video dataset from the 2019–20 challenge, with **over 100,000 video clips** from 3,426 actors, manipulated by various face-swapping and AI techniques. It is the largest public deepfake video dataset to date. | Key for **manipulation detection** – provides a huge diversity of **AI-generated face videos** for training robust models. **Modality:** video (with audio) for deepfake vs real classification. Released for the community via Kaggle; all actors consented. **License/ToS:** Free for research; must agree to usage terms. *Limitation:* Dataset videos are staged and relatively high-quality; TikTok content might be more compressed or edited differently. |
| Celeb-DF ++ | A challenging deepfake dataset of celebrity interview videos. Contains **22 deepfake methods and over 200 or more videos per method**, with realism similar to "in-the-wild" deepfakes. | Enhances the **authenticity labeling** of the project with more **realistic face-swap videos**. Useful to **validate model generalization** on social media-style content (since these fakes are harder to detect). **Modality:** video (face focus). **License:** Available for research (download via GitHub link). *Note:* High-quality fakes mean models trained on easier fakes (like FaceForensics) can be stress-tested. No TikTok-specific content, but relevant for any face-centric misinformation. |

| | | |
|---|---|---|
| **FakeAVCeleb** | An **audio-visual deepfake** dataset with synced fake videos *and* fake voice tracks. Created from real celebrity videos across diverse ethnic groups, then manipulated both in face and in voice (voice cloning and lip-sync). Aims to address multimodal impersonation. | Supports **multimodal fake-content detection** (both **video and audio**). Relevant for TikTok since clips may use synthetic voices or dubbing along with visual fakes. **License:** Released for research. Helps our pipeline catch cases where video is real but audio is AI-generated, or vice versa. *Limitation:* Dataset size is smaller than DFDC; also, celebrity data may differ from typical user-generated TikToks in style. |
| **FakeTT (TikTok Misinformation Videos)** | **Short-video fake news dataset** with TikTok videos. Collected by querying TikTok for 365 known fake-news events from Snopes (2018–2024). After manual verification, it includes **~1,172 "fake" and 819 "real" TikTok videos** covering 286 distinct events. Each entry has metadata like description, verification status, etc. | Provides real TikTok examples with **misleading vs factual content labels** – crucial ground truth for our **claim/verification** component. Covers **multimodal data** (audio, video, on-screen text) aligned with fact-checks. **Relevance:** Directly reflects TikTok's misinformation challenges (contextual lies, miscaptioned videos, etc.). **Access:** Available to researchers. *Note:* Focuses on news/events content; we may need to extend to other domains. |
| **OpenAI Whisper** (ASR) (Tool) | Open-source speech-to-text model by OpenAI. It's pre-trained on 680k hours of multilingual data and achieves accurate automatic transcripts of audio. | Enables **audio transcript** extraction for TikTok videos, feeding our NLP and claim detection modules. Relevant for TikTok's diverse audio (supports multiple languages and accents). Using it ensures we capture spoken claims for fact-checking. **License:** MIT (model and code are free to use). *Considerations:* It's compute-intensive, but can be batched or quantized. |
| **PaddleOCR** (Scene Text OCR) (Tool) | Open-source toolkit for **optical character recognition** in images/videos. Supports detection and recognition of overlaid or scene text in 80+ languages, with robust performance on diverse fonts. | Allows our pipeline to parse **in-video text** (e.g. captions, memes, signs) for analysis. This addresses the **scene/OCR text** schema element. **Relevance to TikTok:** Many TikToks include text overlays or subtitles that might contain claims or context. PaddleOCR can pull that text for our claim matching or sentiment checks. **License:** Apache 2.0 (free for commercial use). *Limitation:* Accuracy can drop on fast-moving captions, so we may need to do frame selection. |

| | | |
|---|---|---|
| **Google Fact Check Tools API** (ClaimReview) (Tool) | An API providing access to the **ClaimReview** database of fact-check articles. It aggregates fact-check entries from dozens of publishers (PolitiFact, Snopes, AFP, etc.), including claim descriptions and verdicts. We can query by keywords or claims to find related fact-checks. | Critical for **claim verification** in our project: once we extract a claim from a TikTok, we can search this repository to see if it's been checked. This links TikTok content to external **fact-check evidence**. **Modality:** text (claims and reviews). **Licensing:** The API is open for public use (no cost), returning published fact-check metadata. *Usage notes:* We must craft queries from video transcripts/OCR text to get relevant results. Many TikTok claims may already have entries in this dataset. However, not all claims will match directly, so we might need NLP to normalize them. |
| **ClaimBuster API** (Tool) | Scores text for **check-worthiness** using the ClaimSpotter (BERT-based) model; simple REST endpoints for sending transcript/overlay text and receiving a score. | Feeds **Claims & Fact-Check Links** dataset with a checkworthy_score, helping prioritize which spans to normalize and send to fact-check search (e.g., Google Fact Check Tools API). Requires API key; triages *what to check*, not truth; combine with ClaimReview search for verdicts. |
| **TikTok Official APIs** (Display & Research) (Tool) | First-party, ToS-compliant access to TikTok data. **Display API** returns public creator profiles and recent/self-selected videos. **Research API** (for eligible academic orgs) provides programmatic access to public content/accounts for research. | **Best-effort compliant ingestion** that maps to **Content & Exposure** dataset (video_id, caption, hashtags, creator_id, timestamps). Use Display API for app-authorized pulls; apply to Research API if you qualify (eligibility/limits apply). Pair with user-consented exports when needed. *Limitation:* access may be restricted; Research API is for qualifying researchers and has scope limits. |

# Data-Processing Pipelines

## Data Schemas

1. **FaceForensics++ Raw Dataset Input Schema**
   **Purpose**: Enumerate original/manipulated videos with derived attributes for downstream labeling.

```
{
  "dataset": "FaceForensics++",
  "downloaded_videos": list[000.mp4],
  "original_sequences": {
    "youtube": {
      "c0_raw": list[000.mp4],
      "c23_hq": list[000.mp4],
      "c40_lq": list[000.mp4]
    },
    "actors": list[000.mp4]
  },
  "manipulated_sequences": {
    "deepfakes": list[000.mp4],
    "deepfake_detection": list[000.mp4],
    "face2face": list[000.mp4],
    "faceswap": list[000.mp4],
    "neural_textures": list[000.mp4]
  }
}
```

2. **Celeb-DF++ Raw Dataset Schema**
   **Purpose:** Normalize real vs multiple synthetic categories for training/eval.

```
{
  "dataset": "Celeb-DF++",
  "real_videos": {
    "celeb_real": list[celeb_real_000.mp4],
    "youtube_real": list[youtube_real_000.mp4]
  },
  "synthetic_videos": {
    "face_swap": {
      "celeb_df": list[000.mp4],
      "blend_face": list[000.mp4],
      "ghost": list[000.mp4],
      "hifi_face": list[000.mp4],
      "in_swapper": list[000.mp4],
      "mobile_face_swap": [list[000.mp4],
```

```
      "sim_swap": [list[000.mp4],
      "uni_face": list[000.mp4]
    },
    "face_reenact": {
      "da_gan": list[000.mp4],
      "fsrt": [list[000.mp4],
      "hyper_reenact": list[000.mp4],
      "lia": list[000.mp4],
      "live_portrait": list[000.mp4],
      "mcnet": list[000.mp4],
      "tpsmm": list[000.mp4]
    },
    "talking_face": {
      "ani_talker": list[000.mp4],
      "echo_mimic": list[000.mp4],
      "ed_talk": list[000.mp4],
      "float": list[000.mp4],
      "ip_lap": list[000.mp4],
      "real3d_portrait": list[000.mp4],
      "sad_talker": list[000.mp4]
    }
  },
  "testing_list": list[000.mp4]
}
```

3. **DynamoDB Metadata Schema (After ETL)**
   **Purpose:** Canonical metadata row per source asset, post-ETL and ready for training/eval joins.

```
{
  "TableName": "deepfake-dataset-metadata",
  "KeySchema": [
    {
      "AttributeName": "dataset_id",
      "KeyType": "HASH"
    },
    {
      "AttributeName": "item_type",
      "KeyType": "RANGE"
    }
  ],
  "Item": {
    "dataset_id": string,
```

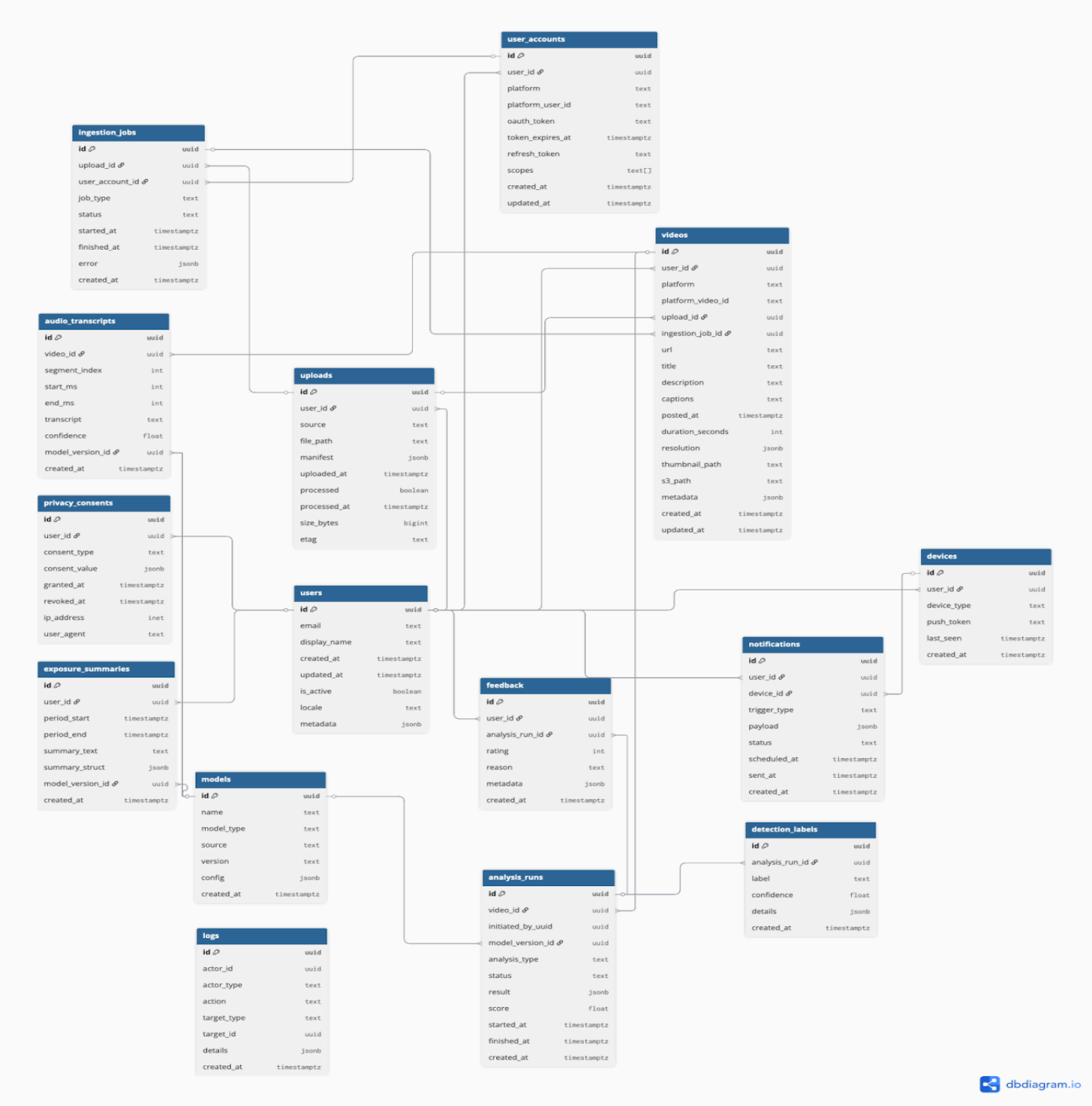
```
    "item_type": string,
    "dataset_name": string,
    "manipulation_method": string,
    "compression_level": string,
    "is_fake": boolean,
    "local_path": string,
    "s3_path": string,
    "file_size": int,
    "processing_status": string,
    "created_at": datetime,
    "updated_at": datetime,
    "tags": ["original", "youtube", "c23"]
  }
}
```

4. **S3 Data Lake Structure**
   **Purpose:** Predictable paths for raw to processed to curated, enabling reproducibility and partition pruning.

```
{
  "s3_bucket": "deepfake-detection-data",
  "structure": {
    "datasets": {
      "faceforensicspp": {
        "raw": list[000.mp4],
        "processed": {
          "faces": {
            "000_003": list[frame_00.jpgs]          },
        "curated": {
          "training": "training.zarr",
          "validation": "validation.zarr",
          "testing": "testing.zarr"
        }
      },
      "celebdfpp": {
        "raw": ...,
        "processed": ...,
        "curated": ...
      }
    },
    "models": {
```

```
      "checkpoints": [],
      "artifacts": []
    }
  }
}
```

## Application ER Diagram



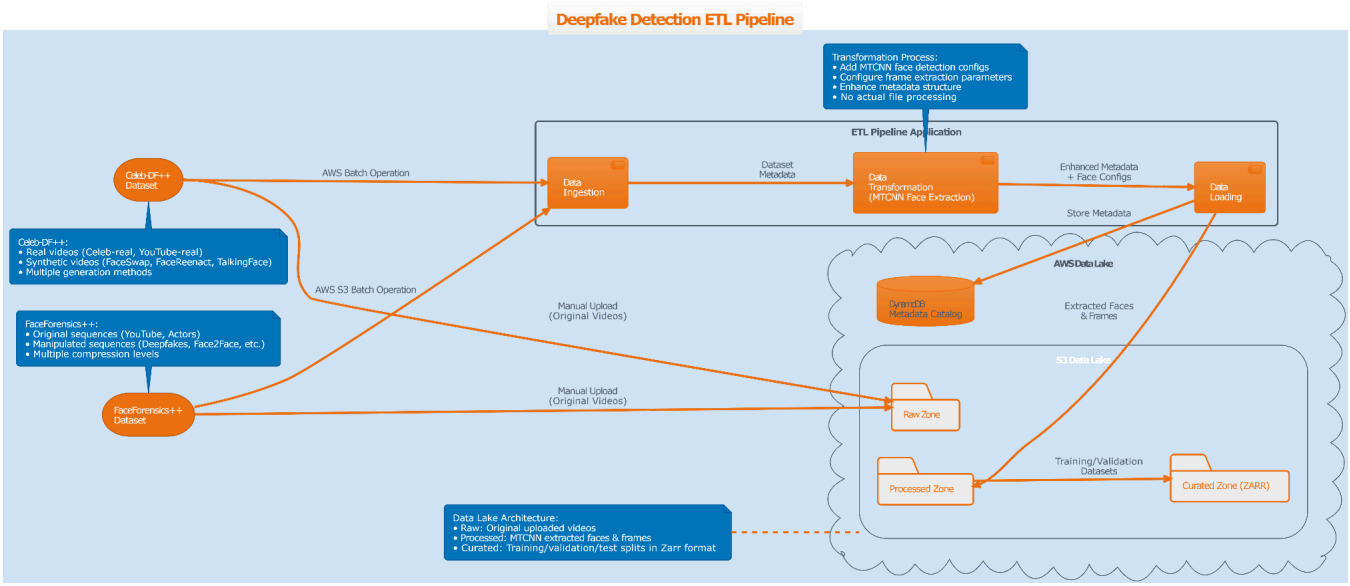
This schema is designed to reflect Anchor, a full-stack ML-powered mobile app for analyzing social media content to detect misinformation and AI-generated content. At its core, the schema centers on the **users** table, which stores the identity and profile data, and branches into related domains: **accounts** (social platforms linked via OAuth), **devices** (push notifications), **uploads** and **ingestion jobs** (media imported either directly or through linked accounts), and **videos** (unit of analysis).

From videos, the schema expands into **audio transcripts**, **analysis runs**, **detection labels**, and **models**, which collectively capture how ML models evaluate the content, what results they produce, and which models were applied. Other supporting structures like **feedback**, **exposure summaries** (user exposure to high-risk content), and **notifications** ensure a full feedback and engagement loop. **Logs** capture system actions for auditability and traceability, while **privacy consents** ensure compliance with data regulations.

The user behaviour is represented across this schema. For example, linking **user_accounts** and **videos** shows what platforms a user consumes content from; **uploads** and **ingestion_jobs** reflect active user submissions and background polling; **analysis_runs** track when content is analyzed and with what confidence; **feedback** captures how users respond to the system's classifications; and **exposure_summaries** quantify trends in risky content exposure over time. Combined, these tables support the operational flow of ingestion, analysis, and feedback along with enabling insights into risk exposure, and engagement patterns.

## Pipeline Diagram

Deepfake Detection Training Pipeline



# When Will The Pipelines Run?

## Training Pipelines

The pipeline is supposed to run once when we first create the required split for model training. Subsequently whenever there is an addition of datasets we will use. We balance scalability (handling massive datasets) with flexibility (supporting both broad and targeted training). The pipeline's uses come together around four main workflows:

- **Scalable training:** AWS S3 buckets, AWS S3 batch operations, and AWS EC2 allows processing of thousands of videos in parallel, enabling production-level deepfake detection model training.
- **Automated preprocessing:** Faces are extracted with MTCNN and metadata is cataloged in DynamoDB, keeping datasets clean and queryable.
- **Targeted model building:** Flexible queries allow the training on detectors on specific manipulation methods (e.g., Face2Face, FaceSwap) or balanced real/fake splits.

## Inference Pipelines

We balance **responsiveness** (quick answers while interest is high) with **accuracy and cost control** (heavier analysis off the critical path). Four pipelines cover the core moments of use: (1) instant checks on a single video, (2) short session summaries, (3) a high-fidelity daily pass, and (4) continuous learning from user feedback. Together, they keep the dashboard fresh and make explanations trustworthy.

| Pipeline | Trigger | Primary use case | Processing |
|---|---|---|---|
| **On-Demand Single-Video Scan** | User shares/pastes a TikTok – runs immediately | Instant verdict + short explanation for one video | Fetch media & minimal metadata → fast ASR on audio → keyframe OCR → quick deepfake screen → extract/normalize claim → fact-check lookup → store results & explanation |
| **Session Batch** | End of session or every 30-60 min | Keep the dashboard updated without heavy cost | Collect last N watched video IDs → prioritize by dwell/watch time → run ASR/OCR on top K → risk scoring → update dashboard cards |
| **Daily Full Analysis** | Nightly (e.g., 2 am local) | High-accuracy pass + exposure analytics and digest | Full ASR; scene segmentation; richer OCR; deepfake models; re-query fact-checks; refresh metrics → generate daily digest & update flags/explanations |

| | | | |
|---|---|---|---|
| **User Feedback Ingest** | On event + hourly aggregation | Close the loop; improve explanations and thresholds | Ingest thumbs/notes → join to items/claims → compute feedback KPIs → queue examples for retraining → adjust risk thresholds/explanation templates as needed |

## Initial Pipeline Code Description

**Note:** The current ETL Pipeline is not completely tested for correct workflow. The datasets we chose to work with CelebDF++ was 54 GB compressed (zip), and about 80 GB decompressed. This increases further when we extract frames from the videos for our use. Similarly the FaceForensics++ dataset was 20 GB compressed, and 40 GB decompressed and all extracted images as pngs would require significantly more space. This is why we decided to host the data on AWS and wait until then. Since the downloads were as zip files we weren't able to download a subset of data and test on that. We did however test a local pipeline with the same transformation which parses through the same structure as the datasets and saves the end product locally.

**Code Implementation:** The ETL pipeline operates through a structured workflow that begins with initializing an `ETLPipeline()` instance and configuring cloud storage using `configure_cloud_storage()` to connect to AWS resources (DynamoDB table "deepfake-dataset-metadata" and S3 bucket "deepfake-detection-data"). The pipeline registers dataset-specific importers (`import_faceforensicspp`, `import_celebdfpp`) that scan dataset structures, transformers (`extract_frames`, `default_transformer`) that process metadata and add configurations, and loaders (`save_to_disk`, `default_loader`) that handle output operations.

The core functionality is executed through `run_cloud_etl()` which orchestrates the entire process by calling the registered importer to scan dataset structure, applying transformers to enhance metadata with processing configurations like frame extraction rates, and then storing results in DynamoDB while uploading files compressed as ZARR to S3 based on the `upload_files` parameter. Additionally, the pipeline provides data retrieval through `download_from_cloud()` for downloading processed datasets from S3 to local directories, creating a complete cloud-native workflow for managing deepfake detection training data at scale.

## Next Steps

The next step is to integrate the AWS services with the existing pipeline, ensuring smooth data flow and scalable storage. The AWS integration code is already in place; the focus will be on connecting our ingestion and upload processes with S3 buckets for secure, durable, and cost-efficient storage. This involves configuring bucket policies, IAM roles, and lifecycle rules so that uploaded TikTok videos, transcripts, and intermediate analysis results can be stored and retrieved seamlessly. We will first commit to a tiny subset plan (e.g., 10 videos from DFDC or a few public TikToks) and a local/dev mode that runs ASR - OCR - store on those files, writing to a local "mock S3" path. Then follow with actual AWS integration. By doing this early, we ensure the pipeline can handle large datasets without bottlenecks, setting a strong foundation for training, inference, and user-facing features.

Following storage integration, we will move towards building an inference pipeline. This means fetching user data (videos, audio, metadata) from TikTok or user uploads, applying trained models to generate predictions, and returning results in real-time to the mobile app. To extend capabilities, we will also integrate fake news detection APIs, enabling the system to cross-check video transcripts and metadata against external knowledge bases and detection services. This hybrid approach, our in-house models plus third-party APIs, provides higher accuracy and coverage, especially in early stages when datasets are still maturing. Together, these steps bring the system closer to an end-to-end solution that not only ingests and stores media but also delivers actionable insights back to users.